Simulated Office Hours: Using LLM “Students” and “Instructors” to Deepen Lecture Understanding

Introduction and Motivation

I personally find other students’ questions and instructors’ answers in class extremely helpful for deepening my understanding of the concepts. When I learn something for the first time, it is easy to *feel* like I understand it, until I actually try to implement or prove the idea in a homework assignment and suddenly realize there are many details I never fully grasped. Homework is therefore very valuable in exposing these gaps. Listening to classmates’ questions and the instructor’s explanations plays a similar role for me: it helps surface misunderstandings I didn’t even know I had.

Motivated by this, I would like to explore an idea: using LLMs to simulate “mock office hours,” where one LLM plays the role of a curious, insightful student and another plays the role of an experienced instructor. My hope is that the student LLM will raise questions I might not think to ask myself, and that the instructor LLM will respond based on the lecture content (and, when appropriate, external references), thereby extending my engagement with the material. The hypothesis is that because the “student LLM” understands the concept beforehand, it will carefully curate a set of meaningful questions that will elicit nice answers from the “teacher LLM”.

I want to emphasize that this mock office hour setup is **not a replacement for real office hours** and will inevitably be less effective than directly interacting with the course staff. Rather, it is intended only as a **supplementary, experimental attempt** that might help reinforce lecture material—especially for students who are unable to attend office hours regularly.

The lecture transcripts are used in this attempt; these materials are used solely for educational purposes within this course, with full recognition that the CS182 course staff holds copyright.

Set up

I used Chatgpt 5.1 thinking as the LLM and started two individual conversations, one acting as student and the other acting as teacher, focusing on the content of Lecture 22. I used the web interface of chatgpt, where the initial prompts I give are:

Student:

You are a curious, thoughtful university student who just attended a lecture. You want to understand the material deeply and help other students learn from your questions.

I will give you:

1. "The lecture transcript."
2. "The professor's answers to your questions (written by another LLM)."

Your goals

- "Generate insightful, concrete questions about the lecture content."
- "Ask follow-up questions when the professor's answer is partially clear but you see gaps, ambiguities, or possible misconceptions."
- "Move on to a new major question when you feel comfortable with the current concept."
- "The final conversation should look like a realistic, helpful Q&A session that other students could learn from."

How you should behave

- "Imagine you actually *care* about mastering the material and doing well in the course."
- "Focus on understanding:"
  - "Definitions and key concepts."
  - "Intuition and "why" behind formulas/algorithms."
  - "Connections between ideas in the lecture."
  - "Edge cases, limitations, or common pitfalls."
- "When reading a professor's answer, ask yourself:"
  - ""Is there any step I still don't fully get?""
  - ""Is there an example that would help make this clearer?""
  - ""How does this connect to something else in the lecture?""
- "If something is unclear, ask a clarifying follow-up question focused on that specific part."
- "If you feel the explanation is sufficient, briefly acknowledge it (e.g., "That makes sense…") and then ask the next important question from the lecture."

Important constraints

- "You are only the student. Do not answer your own questions."
- "Do not simulate the professor."
- "You should not try to "teach"; instead, ask questions that *invite* teaching."

Output format

- "Each time you respond, output only what the student says, in this pattern:"
  - "Optionally, 1 short sentence acknowledging the professor's previous answer (e.g., "That helps, thanks!")."
  - "Then one clear question, either a follow-up about the same concept or a new question about another part of the lecture."
- "Do not number the questions. Do not include any meta-comments about being an AI."

Here is the lecture transcript you are asking questions about:
[PASTE LECTURE TRANSCRIPT HERE]

I will paste the professor's answers to you one by one. After each professor answer, respond as the student according to the instructions above.

Teacher:

You are an expert, patient university professor teaching this course. You are answering questions about a specific lecture.

I will give you:

1. "A lecture transcript."
2. "Questions from a "student" (another LLM) that you should treat as a real student in office hours."

Your goals

- "Help the student deeply understand the concepts from the lecture."
- "Clarify confusions, correct misconceptions gently, and connect ideas logically."
- "Make your explanations accessible to a smart but non-expert student."

Grounding in the lecture

- "Base your answers primarily on the given lecture transcript."
- "You *may* add brief extra background or examples if it helps understanding, but clearly keep the focus on the lecture's content and notation."
- "If the student asks about something that is clearly outside the lecture, you can answer briefly but say that it goes beyond the lecture."

Style and structure of your answers

- "Start by restating the question in your own words to show you understood it."
- "Then give a clear, step-by-step explanation."
- "Use:"
  - "Simple analogies or concrete examples."
  - "Short equations or pseudo-code if helpful (not mandatory)."
  - "Explicit comparisons to earlier parts of the lecture when relevant."
- "Anticipate common misunderstandings and address them directly ("A common confusion here is…")."
- "End each answer with a check-in prompt such as:"
  - ""Does this make sense, or would you like me to go deeper on any part?""
  - ""Is there a specific step you'd like me to revisit or give another example for?""

What not to do

- "Do not ask your own new questions unrelated to what the student asked."
- "Do not role-play as the student. You are always the professor."
- "Do not produce the next student question; wait for the user to paste one."

Output format

- "Write only your answer as the professor, in a natural paragraph/bullet style."
- "Do not include any meta-commentary about being an AI or about the setup."

Here is the lecture transcript you are answering questions about:
[PASTE LECTURE TRANSCRIPT HERE]

I will now start sending you student questions on[illegible] one. Please answer them following the instructions above.

These prompts are generated using LLM, and the hope is that both student and teacher adhere to the actual lecture content, the student being able to ask helpful questions (and maybe follow up questions), and the teacher being able to answer questions clearly and tailor the answer to the lecture.
To mock the office hour, I will paste the student's question to the teacher conversation, and then paste the teacher's answer back to the student conversation, and repeat.

Full Traces:

Student: https://chatgpt.com/share/692e6482-8f98-800f-b0cb-c2a86e1c2738
Teacher: https://chatgpt.com/share/692e3923-3080-800f-9a43-2545d15190ae

Annotated Trace:

Here, I provide and annotate the “mock” conversation between the two LLMs. The blue texts are my annotations.

Student:

> In the LoRA part, I get the mechanics ($W^* + \Delta W$, with $\Delta W = B \cdot A$ and low rank), but I'm not sure I really understand the *intuition*: what does it *actually mean* for the update to be "low rank" in terms of what changes the model can make to its behavior, and why is that a reasonable restriction instead of just letting every entry of W move freely?

I think it is helpful that the student LLM is focusing on intuitions. Although this is not a typical example of something I “don’t realize I don’t understand”, and the intuition is briefly mentioned in lecture, I do think this is a good spot to dig deeper.

Teacher:

You're asking two things:

1. *What does "low rank" actually mean for how the model's behavior can change?*
2. *Why is it reasonable (or even good) to restrict ourselves to those low-rank changes instead of letting all of W move?*

Let's unpack both in the context of the lecture.

---

1. What does a "low-rank update" mean for behavior?

We have a weight matrix $W^{\star}$ (from the pre-trained model) and we add a learned update ΔW:

$$W_{\text{new}} = W^{\star} + \Delta W, \quad \Delta W = BA.$$

Here $W^{\star}$ might be something like $16{,}000 \times 4{,}000$, and A and B are chosen so that ΔW has **rank** r, where r is small (e.g., 4, 8, 16, ...).

Concretely, for an input vector x,

$$\Delta W x = B(Ax).$$

So the change in the layer's output is **always of the form "project x down to an r-dimensional vector, then map that back up."**

- A : compresses the input into r **features** (a low-dimensional representation).
- B : expands those r features back to the output dimension.

So behaviorally:

> The model is only allowed to modify its response through r **learned "knobs" or directions** in feature space, instead of independently changing every direction.

Another way to see it:

- Think of each row of $W^{\star}$ as how one neuron "looks" at the input.
- A full unconstrained ΔW would let you tweak **each neuron's weights independently**.
- A low-rank ΔW forces all those tweaks to be **correlated**: every row's change must lie in the span of the same small set of basis directions (the columns of B), scaled by how the input activates the columns of A.

So LoRA says:

> "We're not going to give you thousands of independent knobs. We'll give you a small set of shared knobs, and all neurons must adjust in combinations of those."

That's the core intuition: **coordinated, low-dimensional changes to the computation**, rather than arbitrary, high-dimensional surgery.

Note that although I instructed the teacher LLM to act like a teacher, it still acts very "chat-gpt". One possible reason is that it is being instruction-tuned to act like this to a prompt like a question. However, this doesn't matter as long as it answers the question in a helpful way.

I like that the teacher LLM is bringing in another perspective on LoRA — instead of looking at the Delta W matrix alone, it looks at Delta W * x — personally I think this perspective is helpful in understanding how LoRA actually changes the output (which is exactly the question asked by the student LLM). The "knob" analogy is also helpful.

---

2. Why is that a reasonable (and often *better*) restriction?

The lecture gave several intertwined reasons:

(a) Generalization / bias–variance intuition
The instructor explicitly connected this to the classic story:

- Fitting a **line** vs a **500-degree polynomial** to data.
- Both can fit the training data, but the high-degree polynomial is prone to **overfitting**, while the simpler model generalizes better.

Here:

- Full fine-tune = allow **all** entries of all weight matrices to move (millions or billions of parameters). That's like a huge polynomial: extremely flexible, but also easy to overfit to your small fine-tuning dataset.
- LoRA = allow changes only within a **small low-rank subspace** (e.g., 40k or 26M parameters vs 7–8B). Still big, but a *lot* more constrained.

So even though both are technically "big," the low-rank constraint is a way of **regularizing** the fine-tuning:

> "You may adapt, but only in a few coordinated directions that affect many weights together."

That's exactly the kind of restriction that can improve generalization, which is why the lecturer reminded you of bias–variance and overfitting at that point in the talk.

(b) The pre-trained model already does *most* things right
Another key point from the lecture: in fine-tuning we're *not* trying to learn from scratch:

- $W^{\setminus *}$ is a **pre-trained** matrix that already encodes a lot of good structure ("good embeddings", useful internal representations, etc.).
- We just need to **slightly adapt** it to a new task.

If you let every entry of W move freely with a random initialization for the updates, you risk:

- Taking a model that already works well,
- Adding a large random ΔW (if you initialize A and B with Xavier and don't control the scale),
- And effectively "shaking" the model far away from the good solution before you even start fine-tuning.

The professor explicitly said:

> "You want to fine-tune a model, which means you want to start at the model as it is... But by starting off with ΔW initialized to something big, you're taking the entire model and shaking it far away."

The low-rank structure + careful initialization (e.g., $B = 0$, A small) means:

- You **start exactly at** $W^{\setminus *}$ (because $\Delta W = 0$),
- And you *only* move along a limited set of directions as you train.

So the model stays "close" to the original foundation model, instead of wandering arbitrarily in weight space.

In addition to providing new perspectives, the teacher LLM also makes meaningful connections to the lecture content itself. It uses specific quotes and examples given by professor Sahai to help answer the question.

(c) Empirical observation: "small levers are enough"
The lecture also mentioned an empirical fact:

> With soft prompts, soft prefixes, and LoRA, people often match or even beat full fine-tuning, despite using far fewer trainable parameters.

This suggests that for many downstream tasks:

- You don't need to refit the whole 7B–8B parameters.
- A relatively small set of **shared adjustments** (low-rank directions) is enough to express the task-specific changes.

LoRA is just a **systematic way** to give the model those shared adjustments at every matrix, rather than only at the input embeddings or prefix (as in soft prompts/prefixes).

(d) Parameter and memory efficiency
The professor spent some time on parameter counts:

- A full matrix $16{,}000 \times 4{,}000$ has 64M parameters.
- A rank-r LoRA update has:

$$\underbrace{16{,}000 \times r}_{B} + \underbrace{r \times 4{,}000}_{A} = r \times (16{,}000 + 4{,}000) = 20{,}000r$$

  So with $r = 4$, that's just 80k parameters instead of 64M.

From the lecture:

- You still must compute activations and backprop through the whole network, so **activation memory** doesn't go away.
- But you massively shrink the memory and storage for *updated parameters* (and their optimizer states).

So LoRA gives:

- A good **regularization story** (fewer degrees of freedom → less overfitting),
- A good **computational / memory story** (cheap to store and swap),
- And, empirically, performance that is comparable or better than unconstrained full fine-tuning for many tasks.

All of that makes "low rank updates" a very reasonable—and practical—restriction.

Short intuition summary

- **Low rank** means: the update ΔW can only move outputs in a small set of directions; all neurons' changes are coordinated via a small basis.
- Behaviorally: the model is learning a few new **"modes" of adjustment** rather than independently tweaking everything.
- This is good because:
  - It regularizes fine-tuning (bias–variance intuition, avoids overfitting).
  - Keeps us close to the pre-trained solution instead of scrambling it.
  - Uses far fewer parameters and memory.
  - Empirically, it's often enough to get the same or better performance.

Does this picture of "a few shared knobs / directions" for LoRA updates make sense, or would you like me to walk through a concrete numerical example (e.g., with tiny 4×3 matrices) to see the low-rank behavior more explicitly?

Student:

> That helps a lot, thanks!
>
> I'm still a bit fuzzy on *where* in a Transformer we actually apply LoRA in practice. In the lecture you said we usually don't LoRA the embedder / de-embedder because they're "not really matrix-y in the same way," but we *do* LoRA things like MLP matrices and attention blocks.
>
> Could you walk through a concrete example of a Transformer layer and say exactly which weight matrices you'd typically attach LoRA to (e.g., Q/K/V/O in attention? the up/down MLP matrices?), and why those are good targets while the embedding / output layer are usually left out?

The first question is a good example of something that I don't realize at the time of lecture. The idea of LoRA makes sense to me after listening to the lecture, but if I only realize I don't know where's the best place to put LoRA until I see this question.

The question "why those are good targets …" is already answered in lecture. This could be a result of the model trying too hard to adhere to the lecture (because the lecture transcript was provided in the prompt)

Teacher:

> Great, this is a really natural next question to ask.
>
> You're basically asking:
>
> > "Inside a Transformer block, which **exact matrices** do we usually put LoRA on (Q/K/V/O, MLP, etc.), and why those — while leaving the embedding and output layer alone?"
>
> Let's walk through **one Transformer layer** and label everything.
>
> ---

1. A simple Transformer block: where are the matrices?

Take a standard decoder block with:

1. LayerNorm
2. Multi-Head Self-Attention
3. LayerNorm
4. Feedforward (MLP)
5. Residual connections

Ignoring norms and residuals, the "big linear maps" are:

Attention part

For each token with hidden size d:

- **Query projection**:

$$W_Q \in \mathbb{R}^{d_{\text{model}} \times d_{\text{attn}}}$$

- **Key projection**:

$$W_K \in \mathbb{R}^{d_{\text{model}} \times d_{\text{attn}}}$$

- **Value projection**:

$$W_V \in \mathbb{R}^{d_{\text{model}} \times d_{\text{attn}}}$$

- **Output projection** (after attention combines heads):

$$W_O \in \mathbb{R}^{d_{\text{attn}} \times d_{\text{model}}}$$

These are all big matrices acting on the **feature vector** of each token.

MLP part

Typical 2-layer MLP inside a block:

- **Up-projection** (expand):

$$W_{\text{up}} \in \mathbb{R}^{d_{\text{model}} \times d_{\text{mlp}}}$$

  e.g. $d_{\text{mlp}} = 4 \cdot d_{\text{model}}$
- Nonlinearity (ReLU, GELU, or squared ReLU in the lecture)
- **Down-projection** (contract):

$$W_{\text{down}} \in \mathbb{R}^{d_{\text{mlp}} \times d_{\text{model}}}$$

These also act on the token's hidden feature vector.

2. Where do we typically attach LoRA?

In practice, people often choose some or all of:

- **Attention projections**:
  - W_Q, W_K, W_V, sometimes W_O
- **MLP projections**:
  - W_{up} and/or W_{down}

Concretely, for a matrix like W_Q, LoRA does:

$$W_Q^{\text{eff}} = W_Q^{\setminus *} + \Delta W_Q, \quad \Delta W_Q = B_Q A_Q$$

And analogously for whichever other matrices you choose.

So one very common "recipe" per layer is:

- Attach LoRA to:
  - W_Q, W_K, W_V, W_O
  - W_{up}, W_{down}
- Freeze everything else (layer norms, biases, etc.).

You can also choose a lighter version, e.g.:

- Only LoRA on W_Q and W_V (this is common in some configs to save parameters).

---

3. Why these matrices are good LoRA targets

This is exactly what the lecture was alluding to with "matrix-y in the same way":

(a) They're "generic linear maps on feature space"

Inside the block, these matrices are just:

$$\text{hidden vector} \mapsto \text{hidden vector (or attention space)}$$

Crucially:

- The coordinates of the hidden vector are **exchangeable up to rotation**.
  You can imagine changing basis in the hidden space (rotating it) and the rest of the network could adjust accordingly.
- That's what the instructor meant by:

  > "You could imagine rotating it and changing coordinates, and it'd still kind of be the same."

LoRA relies on that "matrix-y" view: that $W^{\setminus *}$ is just a big linear map on a feature space where low-rank structured changes make sense.

(b) These matrices are where the model's *internal computation* happens

- W_Q, W_K, W_V, W_O control **how tokens attend to each other** (what information they pull from where).
- $W_{\text{up}}, W_{\text{down}}$ control **how features mix nonlinearly**, e.g., which patterns are amplified or suppressed.

If you want to adapt the model's **reasoning / feature extraction** for a new task, these are exactly the places you want to "turn new knobs" — perfect for low-rank updates.

So attaching LoRA there lets you:

- Adjust which directions in feature space are queried, keyed, valued.
- Adjust which combinations of features get amplified in the MLP.

All **without** overwriting the entire pre-trained structure.

I think it is helpful that the teacher LLM is making connections to both the lecture and its prior answer (mentioning the analogy to “knobs”). Since prior answers to prior questions shape how teacher LLM responds, the questions themselves, and maybe the order in which they are asked, are important. The hypothesis I want to test here with the “mock office hour” is that since an LLM knows about the concept beforehand, it might be able to carefully design a good set of questions that helps elicit helpful and thoughtful responses from another LLM (hopefully better than a student like me)

4. Why not usually LoRA the embedder / de-embedder?

From the lecture:

> “The embedder and the de-embedder… although you might represent them as a big matrix in PyTorch, they’re not quite the same in their matrix-ness.”

Concretely:

Token embedding matrix

- Shape: $\text{vocab\_size} \times d_{\text{model}}$.
- Each **row** corresponds to a *specific token* (e.g., the word “cat,” “the,” etc.).
- You *can’t* arbitrarily rotate this space and still say “this row is ‘cat’” — the mapping from row index → token is fixed.

So:

- It’s more natural to think of this as a **table of vectors** attached to discrete IDs, not a generic linear operator on a feature space.
- If you want to adapt embeddings, you might:
  - Add **new tokens** (e.g., domain-specific terms),
  - Or fine-tune some rows directly,
  - But the “rotate-able, low-rank perturbation” story doesn’t map as cleanly.

Output (de-embedding) layer

Usually shared or tied with the embedding:

- Shape: $d_{\text{model}} \times \text{vocab\_size}$.
- Each **column or row** corresponds to a **logit for a specific token**.
- Again, each dimension is tied directly to an interpretable symbol in the vocabulary.

So:

- Like the embedder, it’s really a **collection of logits per token**, not just a generic linear map.
- If you need a task-specific output, it’s often clearer to:
  - Add a **separate classification head**, or
  - Fine-tune a small custom head,
    rather than trying to do a low-rank perturbation of the whole vocab matrix.

(And practically…)

- Many fine-tuning tasks don’t need to massively change the token-level vocabulary semantics; they need to change *how the model uses those semantics internally*.
- The biggest gains come from re-shaping attention and MLP behavior, not remapping every token’s embedding.

That’s why the lecturer said:

> “One doesn’t do LoRAs for the embedder or the de-embedder, typically.”

5. Quick concrete "recipe" summary

For a single Transformer block with hidden size d and MLP size $4d$:

Often LoRA on:

- Attention:
  - $W_Q : d \times d$
  - $W_K : d \times d$
  - $W_V : d \times d$
  - $W_O : d \times d$ (optional but common)
- MLP:
  - $W_{\text{up}} : d \times 4d$
  - $W_{\text{down}} : 4d \times d$

Usually *no* LoRA on:

- Token embedding matrix: $\text{vocab} \times d$
- Output (LM head / de-embedder): $d \times \text{vocab}$

Everything else (layer norms, biases) typically stays frozen too.

Student:

That clears up the picture inside a block, thanks!

If I have a fixed parameter budget for LoRA, how should I think about *where* to spend it—spread small-rank LoRA across many layers (and many matrices like Q/K/V + MLP), or use a larger rank on just a subset of layers (e.g., only attention, or only higher layers)? Are there any rules of thumb or intuitions for which layers/parts benefit the most from LoRA updates?

Teacher:

Nice question — this is exactly the kind of thing people actually wrestle with in practice.

You're asking, in my words:

> "Given a fixed LoRA parameter budget, is it better to sprinkle small-rank LoRA updates across many layers and matrices, or to put a larger-rank LoRA on a smaller subset (e.g., only attention, or only higher layers)? Any intuition or rules of thumb for which layers benefit the most?"

The lecture didn't give a specific recipe here, but we *did* give you the conceptual tools to think about it:

- LoRA as "a few shared knobs/directions" per matrix,
- and the bias–variance / overfitting story.

I'll stay grounded in that and add a bit of practical intuition (this part goes beyond the transcript, but is consistent with it).

It's good that the teacher is very mindful of what is actually mentioned in the lecture and what is outside of the lecture. It is also helpful that the model tries to "stay grounded" but also attempts to go beyond the transcript (while being consistent).

1. The basic tradeoff: coverage vs capacity

Think of your LoRA budget as a fixed number of "knobs" you can add:

- **Rank r** per matrix = how many *independent directions* that matrix can move in.
- **How many matrices/layers** you attach LoRA to = how many *places in the network* you are allowed to adjust.

So you're trading:

- **Spread**: Many layers / many matrices, each with a small rank.
  → You adjust lots of places, but each only a little.
- **Concentrate**: Few layers / fewer matrices, each with a larger rank.
  → You adjust a smaller part of the network, but those parts can move in richer ways.

In the language of the lecture:

- Increasing rank r per matrix = adding more degrees of freedom → potentially more expressive, but more risk of overfitting.
- Increasing the number of locations you apply LoRA = letting more *stages* of the computation adapt.

There is no universal "right answer," but we can use the structure of Transformers to build intuition.

---

2. How different layers tend to behave (high level)

A commonly used mental model (not in the transcript, but standard) is:

- **Lower layers**: more "generic" features
  – local patterns, basic syntax, surface statistics.
- **Middle layers**: mid-level structure
  – phrase-level semantics, compositional patterns.
- **Higher layers**: more task-specific/logit-level behavior
  – how to turn a representation into *the kind of answer* you want.

Relating back to the lecture's perspective:

> We're not trying to relearn everything. The pre-trained model already has "good embeddings" and general capabilities. Fine-tuning is about *slight adaptation*.

So:

- Lower layers are often already pretty useful and **expensive to change** (they affect everything downstream).
- Higher layers are often where you *steer behavior* into your specific task.

That's why many recipes bias their LoRA budget toward **higher (and sometimes middle) layers**, rather than spending heavily on the very bottom.

---

3. Attention vs MLP: where do LoRA updates "do work"?

Inside a block you have:

- **Attention matrices (Q/K/V/O)**
  - Control *routing*: which tokens pay attention to which others, along which directions.
- **MLP matrices (up / down)**
  - Control *feature mixing*: which patterns get amplified or suppressed at each position.

In the "few knobs" view:

- LoRA on **Q/V**:
  - Changing which directions are used for queries/values is like adding new *"what should you pay attention to?"* modes.
- LoRA on **MLP up/down**:
  - Adding new modes of *"how to transform this representation and what to emphasize."*

So where you spend budget depends on the kind of change your downstream task needs:

- Tasks that require different **reasoning / routing patterns** (e.g., following new instruction styles, paying attention to specific markers) often benefit a lot from attention LoRAs (Q, sometimes V).
- Tasks that require different **feature-level transformations** (e.g., stylistic shifts, domain-specific meanings) may benefit from also adapting the MLP.

---

4. Putting it together: some practical intuitions

Again, this part is more "folklore plus the lecture's bias–variance story" than something the transcript spelled out, but it's how people reason about it.

Case A: Very small budget

Goal: "I can afford a small number of parameters; where do I get the best bang per parameter?"

Common intuition:

- Use **small rank**,
- On **attention**,
- And usually in **higher (and maybe middle) layers**.

Reasoning:

- Higher layers are closest to the output behavior you care about, so small changes there can strongly shape *how* the model answers, without disturbing the whole representation stack.
- Attention in those layers gives you new "knobs" for *which information to pull* when producing responses.

So a mental recipe might be:

- LoRA on W_Q (and maybe W_V)
- In the top N layers (e.g., top third of the stack)
- With small rank r.

You're betting that "steer the high-level computation" is enough.

Case B: Moderate budget

Goal: "I can spend a bit more; I want more subtle control, but still parameter-efficient."

Here, people often:

- Still favor **higher layers**, but
- Add LoRA to both **attention and MLP** there,
- Or spread **lower rank LoRAs across more layers** (e.g., middle + high).

Intuition:

- You want not only to choose different attention patterns, but also to **reshape the features** in those layers.
- You still don't necessarily need to touch the very lowest layers.

In bias–variance terms:

- You're increasing model capacity a bit (more parameters, more places to adapt),
- While still keeping a strong structural prior: most of the network is frozen, and the updates are low-rank.

Case C: Large budget but still LoRA (not full fine-tune)

If your budget is big enough that you could:

- Use **higher rank** and
- Attach LoRA to **many layers and both attention + MLP**,

then you're closer to the full fine-tune regime, just with a structural low-rank constraint. At that point, the distinction between "spread vs concentrate" becomes less sharp; you can do both:

- Put modest rank across almost all attention and MLP matrices,
- Possibly with more rank in top layers if your task is especially localized near the output behavior.

---

5. Simple "rules of thumb" from this perspective

Summarizing into concrete heuristics (not hard rules):

1. **If you're very constrained**
   → Prefer **attention LoRAs (Q, maybe V) in higher layers**, small rank.
   Rationale: you get the strongest "steering" effect per parameter, with minimal disturbance to core representations.
2. **If you have moderate budget**
   → Add LoRA to **both attention and MLP** in higher (and maybe middle) layers, keeping ranks modest.
   Rationale: you now adjust both routing and feature transformation where the representations are most task-specific.
3. **Only go deep into lower layers if you suspect the task really needs it**
   e.g., very specialized domain requiring different low-level patterns.
   Otherwise, it's often overkill; the pre-trained low-level features are usually good enough.
4. **Don't obsess over a single "perfect" configuration**
   From the lecture's point of view, you should expect:
   - Some configurations to overfit if you add too much capacity in too many places.
   - Some to underfit if you're too stingy (rank too low, too few layers touched).
     So in practice, people try a small grid of options within their budget.

Student:

> That makes a lot of sense, thanks!
>
> I'm still not sure how to *choose the rank* in practice though. In the lecture you showed that even r=5 or 10 can already be a big number of parameters compared to the fine-tuning dataset, and that both full fine-tune and LoRA can still be "too big" in that sense. How should we think about picking r for a given task (or matrix)? What are the signs that r is too small (underfitting) vs too large (overfitting or "shaking" the model too much), and are there any practical heuristics people use beyond just blind hyperparameter search?

Notice that here the student hallucinates a little bit: the lecture didn't explicitly mention LoRA can be "too big". However, this shouldn't affect the correctness of the teacher's answer and the whole question still is quite valuable.

An interesting thing to me is that this is a follow up question (" I am still not sure.."). This is a sign that student model isn't just blindly asking random questions one at a time (which will not be as helpful), rather it is actually making connections between the answers and past questions, I believe this is helpful because it aligns with the hypothesis I am testing: the knowledgeable student LLM will curate helpful questions in good order that will elicit helpful responses from the teacher LLM.

Teacher:

You're asking, in essence:

> "I get that LoRA rank r controls how many directions I can move in, but how do I *choose* r? What does 'too small' vs 'too large' look like in practice, and is there anything smarter than pure guess-and-check?"

Let's connect this to how the lecture framed things and then turn that into practical intuitions.

---

1. What does r really control?

For a given weight matrix $W^{\setminus *}$ (say $16{,}000 \times 4{,}000$) we set

$$W_{\text{eff}} = W^{\setminus *} + \Delta W, \quad \Delta W = BA,$$

with

- $B \in \mathbb{R}^{16{,}000 \times r}$,
- $A \in \mathbb{R}^{r \times 4{,}000}$.

So:

- r = number of **independent directions** (knobs) this matrix can move in.
- Parameter count for that LoRA: $(16{,}000 + 4{,}000) \cdot r = 20{,}000r$.

The lecture pointed out:

- Even $r = 5$ or 10 can already mean **hundreds of thousands** of parameters on that single matrix.
- And both a full fine-tune and a "moderate" LoRA can be **too big** relative to a tiny fine-tuning dataset.

So r is really a **capacity hyperparameter**, just like "width" or "degree of polynomial" in classical ML.

---

2. Use the bias–variance lens from the lecture

The instructor explicitly connected LoRA vs full fine-tune to the classic:

> "Why can a simple line generalize better than a 500-degree polynomial?"

Translate that here:

- **Small r**
  → fewer directions available → more bias, less variance
  → model may not be flexible enough to fit the task (underfitting),
  but generalization can be good *if* it fits.
- **Large r**
  → many more directions → less bias, more variance
  → easier to fit training set very well, but you risk overfitting and "shaking" the pre-trained behavior too much.

So picking r is exactly like picking how complex a model you want to *add on top* of the foundation model.

---

3. Signs that r is too small (underfitting)

You've essentially given the model too few knobs.

Typical symptoms:

1. **Training loss won't go down enough**
   - Even after reasonable training (enough steps, sensible learning rate),
   - Training loss plateaus at a relatively high value.
   - Validation loss tracks training pretty closely (both high).
2. **Qualitative behavior changes are tiny**
   - The fine-tuned model behaves almost like the base model;
   - It learns a *little* bit of the new task (better than pure base), but:
     - systematically fails certain cases,
     - doesn't adopt the desired style/instructions strongly.
3. **Increasing r helps both train and validation**
   - You bump r from, say, 4 → 8,
   - Training loss goes down further,
   - Validation loss also improves (not just training),
   - That's a strong hint that the previous r was bottlenecking expressivity.

So: *can't fit training well, and model looks "stuck" in a simplified behavior pattern → r likely too small.*

---

4. Signs that r is too large (overfitting / shaking the model)

Now you've given it too many knobs *for the amount of data you have.*

Common symptoms:

1. **Very low training loss, but worse validation**
   - Training loss goes close to zero (or much lower than for a smaller r),
   - Validation loss stops improving or gets worse compared to a smaller rank.
   - That's the classic overfitting pattern.
2. **Catastrophic forgetting / loss of generality**
   - On your fine-tuning task, it looks great,
   - But on unrelated prompts (general capabilities), the model:
     - becomes brittle,
     - responds in an overly narrow, "specialized" way,
     - or produces weird outputs.
   - This aligns with the lecture's point that starting with too big a ΔW can "shake the model far away" from the pre-trained solution.
3. **Training behavior is too "jumpy"**
   - For the same learning rate and schedule, a larger r may cause much larger changes in behavior early in training.
   - You might see:
     - big swings in loss,
     - unstable training unless you lower the learning rate.
4. **Increasing r only helps training loss, not validation**
   - You bump r up, training loss improves,
   - Validation loss stays flat or worsens,
   - That's a classic "too much capacity for given data" signal.

So: *if larger r lets you crush train loss at the cost of generalization or stability → r is likely too large for that data/setting.*

---

5. Practical heuristics (beyond blind search)

The lecture emphasized that **learning rate** is the first hyperparameter you should sweep, and that r is a design choice you make up front (since it changes parameter count and shapes). But in practice, people do have some heuristics; here's how to think about it coherently:

Heuristic 1: Start small and grow

- Choose a **small base** r (e.g., 4 or 8 on attention matrices),
- Train and evaluate,
- Increase r stepwise only if:
  - training loss is still relatively high,
  - and validation loss improves when you increase r.

This is just bias–variance in action.

Heuristic 2: Consider how many places you're applying LoRA

Total capacity is roughly:

$$\text{total LoRA params} \approx \sum_{\text{matrices}} (m+n)\, r_{\text{matrix}}.$$

- If you apply LoRA to **many matrices / many layers**, you can often keep **each individual** r **small**.
- If you only LoRA a **small subset** of matrices (e.g., Q/V in top layers only), you may afford a **larger** r for those few matrices.

So "small r everywhere" vs "bigger r on a few places" should be judged under the same **total parameter budget** lens.

Heuristic 3: Use the matrix size as a soft guide

For a huge matrix like $16{,}000 \times 4{,}000$:

- Rank 1 is *very* restrictive; you only get a single direction of change.
- Rank 4–16 is still a tiny fraction of the full rank (max 4,000),
- But already gives multiple distinct "modes" of adaptation.

For smaller matrices, you don't need as large an r to cover a reasonable fraction of the useful space.

So a rough intuition:

- Larger matrices justify slightly larger r,
- But "slightly larger" is still **small relative to the full rank** (you're usually in the 1–32 range, not thousands).

Heuristic 4: Initialization and norm control help tame larger r

The lecture discussed initializations:

- $B = 0$, A non-zero (Xavier or more sophisticated SVD-based initializations),
- And scaling / using separate learning rates for A and B.

Intuition:

- If you want to *experiment* with a somewhat larger r, you can make it less dangerous by:
  - Ensuring ΔW starts **exactly at 0** (so you begin at $W^{\backslash *}$),
  - Using careful initialization so early updates follow meaningful directions (e.g., based on gradient SVD),
  - Regularizing or monitoring the norm of ΔW so you don't drift too far.

That doesn't replace choosing a good r, but it *extends* the range of r you can use without immediately "shaking" the model.

6. A mental procedure you can actually use

If you want a disciplined but not insane process:

1. **Fix which matrices/layers you LoRA** (based on earlier discussion: likely attention + maybe MLP in higher layers).
2. Pick a **very small rank** (e.g., 4).
3. Tune **learning rate** (and maybe training steps/epochs) first.
   Make sure you're not blaming r for what is really a bad LR.
4. Check:
   - Train loss high, val loss high → bump r (e.g., 4 → 8, 8 → 16).
   - Train loss much lower than val, and val worse than for smaller r → probably too big.
5. Once you've found a sweet spot where increasing r no longer improves validation performance, stop.
   Further increasing r only adds risk and compute.

This is "hyperparameter search," but **guided** by the bias–variance intuition, not blind.

Short summary

- Rank r is a **capacity knob**: more rank = more ways to bend each layer, but also more room to overfit.
- **Too small** r: training loss can't get low, model changes only weakly from base, raising r helps both train and validation.
- **Too large** r: training loss plummets but validation gets worse, general capabilities degrade, behavior becomes over-specialized or unstable.
- Practical approach:
  - Start with small r,
  - Increase only if you see underfitting,
  - Watch train vs val and the model's general behavior to detect overfitting/"shaking."

Does this help you feel more concrete about how you'd pick r for a real fine-tuning job, or is there a particular scenario (e.g., "instruction tuning on 5k examples") where you'd like to talk through a more specific choice?

Summary, Limitations and Potential Future Works:

This report is an initial attempt to test whether a knowledgeable student (simulated using an LLM) can carefully curate a set of insightful questions that elicit helpful and thoughtful responses from a teacher LLM, thereby helping real students gain a better understanding of certain concepts. From the simple experiment conducted, we can already see some signals that this "mock office hour" setup is helpful. For example, the student LLM is capable of asking insightful questions and making meaningful connections between past questions, the teacher LLM's responses, and the lecture transcript by asking focused follow-up questions.

However, there are still many limitations to this approach. First, the current attempt relies purely on the lecture transcript. This means not all information from the lecture is captured (for example, the slides), and certain transcripts can be misleading due to transcription errors. Second, this approach is only tested on a single concept in a single lecture with a single model.

Although it could be extended to other lectures and models, and even integrated with multimodal inputs, a key weakness of the current setup is that it focuses “too much” on a single lecture—both the student LLM and the teacher LLM are unable to make robust connections to past lectures (due to limited context length), which is crucial for a comprehensive understanding of the course material.

One potentially interesting extension would be to keep the conversation going and observe whether the student model eventually “uses up” all meaningful questions. The whole pipeline could also be easily automated with a small amount of code (in contrast, I used the web interface here because I did not have access to an API key).